TechHub

A Unified Technology Product Discovery, Comparison, Deal Aggregation, and PC-Building Platform

---

Project Proposal and Technical Documentation

Prepared by: Your Name

Institution: Your University or Organization

Frontend: React / Next.js with TypeScript

Backend: FastAPI

Database: Supabase PostgreSQL

Document Version: 1.0

Date: October 21, 2026

A platform for discovering technology products, comparing sellers, validating PC-build compatibility, finding nearby stores, and tracking deals from one unified interface.

Abstract

Technology products are distributed across many independent e-commerce websites, physical stores, social-media pages, and digital marketplaces. Customers must repeatedly visit different sources to compare prices, specifications, availability, reviews, warranties, store locations, and promotional offers.

TechHub is proposed as a unified technology-product discovery and comparison platform. It will collect and normalize publicly available or merchant-provided product information, organize offers from multiple verified sellers, and redirect customers to the original seller when they are ready to purchase.

The system will initially focus on PC components and computer-building tools. Its primary differentiator will be a compatibility-aware PC builder supported by a visual PC simulator. Additional features will include local store discovery, price history, deal aggregation, wishlists, price alerts, reviews, and a future gaming marketplace section.

The platform will use React or Next.js for the frontend, FastAPI for backend services, and Supabase PostgreSQL for data storage, authentication, and supporting services.

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Proposed Solution . . . 1

2 Project Vision **2**
- 2.1 Vision Statement . . . 2
- 2.2 Value Proposition . . . 2
- 2.3 Platform Positioning . . . 2

3 Project Objectives **2**

4 Scope **3**
- 4.1 Initial Scope . . . 3
- 4.2 Future Scope . . . 3
- 4.3 Out of Scope for the Initial Version . . . 4

5 Target Users **4**

6 Core Features **5**
- 6.1 Product Discovery . . . 5
- 6.2 Canonical Product Pages . . . 5

6.3 Price Comparison 6
6.4 Price History and Alerts 7

7 PC Builder and Compatibility Engine 7
7.1 PC Builder Workflow 7
7.2 Compatibility Checks 8
7.3 Compatibility Result Levels 8
7.4 Build Scoring 9

8 PC Visualization and Simulation 9
8.1 Purpose 9
8.2 Development Levels 9
8.2.1 Level 1: Two-Dimensional Build View 9
8.2.2 Level 2: Interactive Case View 10
8.2.3 Level 3: Three-Dimensional Simulator 10

9 Local Store Discovery 10
9.1 Store Information 11
9.2 Availability Confidence 11

10 Deals and Offer Aggregation 11
10.1 Supported Offer Types 11
10.2 Offer Information 12
10.3 Offer Collection Policy 12

11 Gaming Section 13

12 Functional Requirements 14
12.1 User Requirements 14
12.2 Merchant Requirements 14
12.3 Administrative Requirements 15

13 Non-Functional Requirements 15

14 System Architecture 17
14.1 High-Level Architecture 17
14.2 Frontend 17
14.3 Backend 18
14.4 Database and Authentication 18
14.5 Background Processing 18

15 Database Design 19

15.1 Core Data Model 19
15.2 Important Data Relationship 20

16 API Design **21**

17 Search and Product Matching **22**
17.1 Search Requirements 22
17.2 Product Matching 22

18 Data Collection and Quality **23**
18.1 Data Sources 23
18.2 Quality Controls 23

19 Merchant Portal **24**

20 Security and Privacy **24**
20.1 Security Measures 24
20.2 Privacy Principles 25

21 Review and Trust System **25**

22 Business Model **26**

23 Implementation Plan **27**
23.1 Phase 1: Foundation and Catalog 27
23.2 Phase 2: Comparison and Price Tracking 27
23.3 Phase 3: PC Builder 27
23.4 Phase 4: Locations and Deals 27
23.5 Phase 5: Visualization 28
23.6 Phase 6: Platform Expansion 28

24 Minimum Viable Product **28**

25 Testing Strategy **29**
25.1 Testing Types 29
25.2 Critical Test Scenarios 29

26 Risks and Mitigation **30**

27 Success Metrics **31**

28 Recommended Team Structure **31**

29 Final Evaluation **32**

30 Conclusion **32**

A Suggested Technology Stack **33**

B Suggested Repository Structure **34**

C Example User Journey **35**

1 Introduction

1.1 Background

Customers looking for laptops, computer components, gadgets, gaming accessories, or digital games frequently encounter several problems:

- Product information is spread across many websites.
- The same product may have different names on different stores.
- Prices and stock availability change frequently.
- Promotional offers are announced through different social-media platforms.
- Product specifications may be incomplete or inconsistent.
- Customers may not know whether PC components are compatible.
- Local store availability is often difficult to confirm.
- Customers may miss discounts because they do not follow every seller.

These problems create a need for a central platform that organizes product information without necessarily becoming a direct retailer.

1.2 Proposed Solution

TechHub will operate as a technology-product aggregation, comparison, and decision-support platform. It will allow users to:

1. Search for technology products from one interface.
2. Compare prices and offers from multiple sellers.
3. Review standardized product specifications.
4. Find nearby physical stores.
5. Build and validate a custom PC.
6. Visualize selected PC components.
7. Monitor price changes and promotional offers.
8. Read reviews and buying guidance.
9. Follow seller links to complete purchases externally.

The platform will not initially process customer payments or manage product delivery. The purchase will be completed on the selected merchant's website or at a physical store.

2 Project Vision

2.1 Vision Statement

> *To create a trusted, unified platform where customers can discover, understand, compare, and visualize technology products before buying them from verified online or local sellers.*

2.2 Value Proposition

The platform's main value proposition is:

> *Find the right technology product, confirm that it meets the user's needs, compare trusted sellers and nearby stores, discover genuine offers, and make an informed purchase decision from one place.*

2.3 Platform Positioning

TechHub should be positioned as a product-discovery and decision-support service rather than as another e-commerce store.

Its strongest initial combination is:

PC Component Catalog + Price Comparison + Compatibility-Aware PC Builder + Local Store Discovery + Verified Deals

3 Project Objectives

The primary objectives are:

- Build a standardized technology-product catalog.
- Connect each product to listings from multiple merchants.
- provide accurate and transparent price comparison.
- Help users understand technical product specifications.
- Detect incompatible PC components.
- Provide a visual representation of a custom PC build.
- Help users discover nearby stores carrying selected products.
- Aggregate verified deals and time-limited offers.
- Provide price histories, wishlists, and price-drop alerts.

- Generate referral traffic for technology retailers.
- Create a scalable foundation for laptops, gadgets, games, and rentals.

4 Scope

4.1 Initial Scope

The first release should focus on PC components and include:

- Processors
- Motherboards
- Graphics cards
- Memory modules
- Storage devices
- Power supplies
- PC cases
- CPU coolers
- Monitors
- Keyboards, mice, and selected peripherals

4.2 Future Scope

After the initial platform becomes stable, it can expand to:

- Laptops
- Mobile devices
- Cameras
- Smart devices
- Networking equipment
- Gaming consoles
- Physical and digital games
- Game-rental services
- Used technology products

- Business technology solutions

4.3 Out of Scope for the Initial Version

The following features should not be part of the first minimum viable product:

- Direct payment processing
- Product delivery management
- A complete three-dimensional model for every PC component
- Real-time inventory without merchant integration
- User-to-user product selling
- Automatic collection from every social-media platform
- Game-account trading
- Support for unverified or unauthorized sellers

5 Target Users

User Type	Needs
General customer	Search for technology products, compare prices, and identify trusted sellers.
PC builder	Select compatible components, estimate costs, and visualize a complete build.
Gamer	Find gaming hardware, evaluate system requirements, and discover game purchase or rental options.
Local shopper	Find nearby physical stores and check whether a product may be available.
Deal seeker	Track discounts, promotional campaigns, bundles, and price reductions.
Merchant	Publish listings, update inventory, promote verified deals, and receive qualified referral traffic.
Administrator	Manage products, merchants, listings, compatibility rules, reviews, and reported content.

User Type	Needs
Content editor	Create buying guides, product descriptions, comparison articles, and expert reviews.

6 Core Features

6.1 Product Discovery

The product-discovery module will provide:

- Keyword and model-number search
- Category navigation
- Brand filtering
- Price-range filtering
- Specification-based filtering
- Seller and location filtering
- Availability filtering
- Sorting by price, popularity, rating, and recency
- Suggestions for misspelled queries
- Related and alternative products

6.2 Canonical Product Pages

Each real-world product should have one canonical product page. Individual store listings should be attached to that product.

For example:

Listing 1: Canonical Product and Merchant Listings

```
Canonical Product:
  ASUS Dual GeForce RTX 5070 12 GB

Merchant Listings:
  Store A | Price: BDT 85,000 | In stock
  Store B | Price: BDT 83,500 | Online only
  Store C | Price: BDT 87,000 | Availability unverified
```

The canonical product page should contain:

- Standardized name
- Manufacturer and brand
- Model number
- Images
- Structured specifications
- Product description
- Advantages and limitations
- Expert and customer reviews
- Seller listings
- Price history
- Related products
- Compatibility information

6.3 Price Comparison

The comparison system should display:

- Product price
- Discounted price
- Original price
- Shipping cost, where available
- Warranty
- Stock condition
- Seller rating
- Listing update time
- Online or physical-store availability
- Affiliate or direct purchase link

The platform should avoid claiming that an item is available unless its inventory has been recently verified.

6.4 Price History and Alerts

Users should be able to:

- View price changes over time.
- Set a target price.
- Receive price-drop notifications.
- Receive back-in-stock notifications.
- Follow a brand, product, category, or merchant.
- Choose email, browser, or in-app notifications.

7 PC Builder and Compatibility Engine

7.1 PC Builder Workflow

The PC builder will guide the user through the following selections:

1. Processor
2. Motherboard
3. CPU cooler
4. Memory
5. Graphics card
6. Storage
7. Power supply
8. Case
9. Monitor
10. Peripherals

For every selection, the system should automatically update:

- Total price
- Estimated power consumption
- Compatibility status
- Relevant warnings
- Alternative products

- Merchant availability

7.2 Compatibility Checks

The compatibility engine should validate at least the following rules:

Check	Validation
CPU and motherboard	CPU socket, chipset, generation, BIOS version, and processor support list.
Motherboard and memory	DDR generation, supported speed, maximum capacity, module count, and ECC support.
Motherboard and case	ATX, Micro-ATX, Mini-ITX, E-ATX, and other supported form factors.
GPU and case	Graphics-card length, thickness, expansion-slot count, and case clearance.
CPU cooler and case	Cooler height, radiator dimensions, fan positions, and case clearance.
CPU cooler and CPU	Supported socket, cooling capacity, and estimated thermal requirements.
Power supply and build	Estimated wattage, recommended headroom, efficiency, and connector availability.
Storage and motherboard	SATA ports, M.2 slots, PCIe generation, and storage interface support.
GPU and power supply	Required PCIe connectors, connector type, and recommended wattage.
Front-panel connectivity	Compatibility between the case's USB ports and motherboard headers.

7.3 Compatibility Result Levels

The system should classify results into three levels:

Error

The selected components cannot work together.

Warning

The components may work, but installation, BIOS updates, thermal performance, power requirements, or physical clearance requires attention.

Suggestion

The build is compatible, but a more balanced or cost-effective component may be available.

Compatibility messages should be written in simple language. For example:

> The selected processor uses the AM5 socket, but the selected motherboard uses the AM4 socket. Select an AM5 motherboard or change the processor.

7.4 Build Scoring

A future build-scoring system may evaluate:

- Compatibility
- Component balance
- Estimated gaming performance
- Productivity performance
- Power efficiency
- Cooling adequacy
- Upgrade potential
- Price-to-performance ratio

The score should be transparent and accompanied by explanations rather than presented as an unexplained numerical value.

8 PC Visualization and Simulation

8.1 Purpose

The PC simulator will help users understand how their selected components fit together. It should support the compatibility engine rather than replace it.

8.2 Development Levels

8.2.1 Level 1: Two-Dimensional Build View

The first version should include:

- A visual case layout
- Selected component images

- Component names and prices
- Compatibility indicators
- Estimated power consumption
- Build summary
- Screenshot and sharing functionality

8.2.2 Level 2: Interactive Case View

The second version may include:

- Component positioning
- Approximate physical dimensions
- GPU and cooler clearance indicators
- Fan and radiator placement
- Cable and connector guidance
- RGB color selection

8.2.3 Level 3: Three-Dimensional Simulator

The advanced version may use Three.js or React Three Fiber to provide:

- Rotatable 3D models
- Zoom and internal case views
- Component insertion animations
- Lighting and RGB customization
- Physical clearance visualization
- Saved visual configurations

The main limitation is the availability of accurate 3D models and dimensions. Therefore, the 3D version should initially support only selected popular cases and components.

9 Local Store Discovery

The location module should allow users to find physical stores near a selected location.

9.1 Store Information

Each store page may contain:

- Store name
- Merchant name
- Address
- Geographic coordinates
- Opening hours
- Phone number
- Website and social-media links
- Available services
- Product listings
- Customer ratings
- Directions
- Last inventory update

9.2 Availability Confidence

Inventory information should use clear confidence labels:

- **Confirmed in stock:** Recently verified through a merchant system or authorized feed.
- **Listed online:** Shown on the merchant website, but local availability is not confirmed.
- **Availability unverified:** The listing has not been recently updated.
- **Out of stock:** The seller has reported that the item is unavailable.
- **Contact store:** The customer should call before visiting.

10 Deals and Offer Aggregation

10.1 Supported Offer Types

The platform may collect:

- Direct product discounts

- Coupon codes
- Bundle offers
- Bank-card discounts
- Mobile-payment offers
- Seasonal promotions
- Student discounts
- Free-delivery campaigns
- Cashback offers
- Social-media promotions

10.2 Offer Information

Each offer should include:

- Title
- Seller
- Description
- Source link
- Start and expiry dates
- Terms and conditions
- Eligible products
- Payment requirements
- Verification status
- Last checked time

10.3 Offer Collection Policy

The preferred data-source priority is:

1. Official merchant APIs
2. Merchant-provided product feeds
3. Merchant dashboard submissions
4. Authorized affiliate feeds

5. Public structured data
6. Permitted website collection
7. Manually verified submissions

Automated collection should respect website terms, robots policies, copyright, data ownership, rate limits, and applicable laws. Social-media offers should retain a link to the original announcement.

11 Gaming Section

The gaming section should be developed after the PC product platform is stable.

It may include:

- Physical game listings
- Digital game listings
- Console and PC editions
- Regional pricing
- Official game stores
- Local retailers
- Verified rental services
- Subscription availability
- System requirements
- User reviews
- Price history

A future feature may compare a game's system requirements with the user's saved PC build.

For example:

> Your saved build meets the recommended requirements for this game and is expected to provide a suitable 1080p gaming experience.

Unauthorized account sales and suspicious game-key marketplaces should not be treated as equivalent to verified official sellers.

12 Functional Requirements

12.1 User Requirements

The system shall allow users to:

FR-U1: Register and sign in.

FR-U2: Browse products without an account.

FR-U3: Search using product names and model numbers.

FR-U4: Filter products using structured specifications.

FR-U5: Compare merchant listings.

FR-U6: view product price history.

FR-U7: Save products to a wishlist.

FR-U8: Create product-price alerts.

FR-U9: Create and save PC builds.

FR-U10: Receive compatibility feedback.

FR-U11: Share a PC build using a public link.

FR-U12: Find nearby stores.

FR-U13: Read and submit product reviews.

FR-U14: Report inaccurate listings or reviews.

12.2 Merchant Requirements

The system shall allow approved merchants to:

FR-M1: Create or claim a merchant profile.

FR-M2: Manage store locations.

FR-M3: Submit listings manually.

FR-M4: Upload product listings using CSV files.

FR-M5: Connect an approved API or feed.

FR-M6: Update prices and stock information.

FR-M7: Publish promotional offers.

FR-M8: View referral-click statistics.

FR-M9: Respond to listing-quality issues.

12.3 Administrative Requirements

The system shall allow administrators to:

FR-A1: Manage users and roles.

FR-A2: Approve merchants.

FR-A3: Create and update canonical products.

FR-A4: Match merchant listings to canonical products.

FR-A5: Merge duplicate products.

FR-A6: Review compatibility data.

FR-A7: Moderate reviews and reports.

FR-A8: Verify offers.

FR-A9: Monitor data-import jobs.

FR-A10: Suspend inaccurate or abusive sellers.

FR-A11: View platform analytics.

13 Non-Functional Requirements

Requirement	Description
Performance	Common pages should load quickly, and frequently accessed results should be cached.
Scalability	The system should support increasing numbers of products, merchants, prices, stores, and users.
Availability	Core search and comparison features should remain accessible during background imports.
Security	Authentication, authorization, validation, encryption, rate limiting, and audit logging should be implemented.
Privacy	Only required user information should be stored, and users should be able to manage their saved data.

Requirement	Description
Accessibility	The interface should follow WCAG principles and support keyboard navigation, labels, contrast, and assistive technologies.
SEO	Public product, category, comparison, deal, and guide pages should provide server-rendered metadata and structured information.
Maintainability	The application should use modular services, typed interfaces, automated tests, and documented APIs.
Transparency	The system should display listing sources, update times, sponsored status, and availability confidence.
Responsiveness	The application should work effectively on mobile, tablet, and desktop screens.

14 System Architecture

14.1 High-Level Architecture

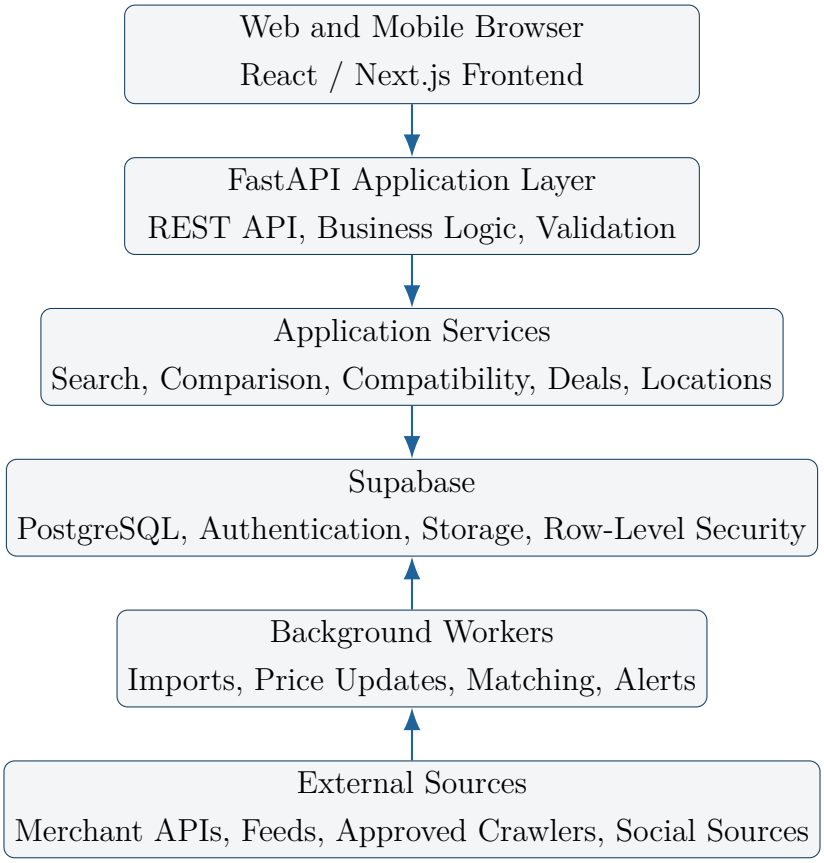


Figure 1: Proposed high-level system architecture

14.2 Frontend

The recommended frontend technologies are:

- Next.js with React and TypeScript
- Tailwind CSS or another reusable design system
- TanStack Query for server-state management
- Zustand or Redux Toolkit for PC-builder state
- React Hook Form for form management
- Mapbox or Google Maps for location features
- Recharts or Chart.js for price-history visualization
- React Three Fiber for future 3D visualization

Next.js is recommended because product and comparison pages require good search-engine visibility, metadata, structured content, and fast initial page delivery.

14.3 Backend

FastAPI will be responsible for:

- Product and listing APIs
- Product normalization
- Search orchestration
- Compatibility validation
- Price comparison
- Store-location queries
- Merchant operations
- Review moderation
- Deal management
- Affiliate-click tracking
- Administrative operations

14.4 Database and Authentication

Supabase will provide:

- PostgreSQL database
- User authentication
- File and image storage
- Row-Level Security
- User session management
- Optional real-time updates

Critical business logic should pass through FastAPI instead of relying only on direct frontend database access.

14.5 Background Processing

Background jobs should handle:

- Product feed imports
- Price and inventory refreshes
- Listing normalization
- Duplicate detection
- Offer expiry
- Price-history recording
- Alert generation
- Broken-link detection

Possible technologies include Celery, Dramatiq, or ARQ with Redis.

15 Database Design

15.1 Core Data Model

A canonical product must be separated from merchant-specific listings.

Table	Purpose
users	Stores application-specific user profile data linked to Supabase authentication.
roles	Defines customer, merchant, editor, moderator, and administrator roles.
brands	Stores technology-product brands and manufacturers.
categories	Stores hierarchical product categories.
products	Stores canonical product identity and general information.
product_variants	Stores model, color, capacity, region, or revision variations.
specification_definitions	Defines valid specification names, units, and data types.
product_specifications	Stores structured specification values for products.
merchants	Stores seller identity, verification, and business information.
store_locations	Stores addresses, coordinates, hours, and contact information.

Table	Purpose
merchant_listings	Connects merchant-specific product listings to canonical products.
inventory_snapshots	Stores time-based stock information.
price_history	Stores historical prices for merchant listings.
deals	Stores discounts, promotional campaigns, and coupons.
deal_sources	Stores source URLs and verification information for deals.
reviews	Stores approved customer or expert reviews.
wishlists	Stores user-created product collections.
price_alerts	Stores notification conditions for selected products.
saved_builds	Stores user-created PC builds.
saved_build_items	Stores selected components within a build.
compatibility_rules	Stores structured rules for component compatibility.
component_dimensions	Stores product dimensions and clearance-related measurements.
affiliate_clicks	Stores outbound seller-link events for reporting and attribution.
import_jobs	Stores data-ingestion job status and error information.
audit_logs	Stores sensitive administrative and merchant actions.

15.2 Important Data Relationship

The recommended relationship is:

Listing 2: Simplified Product Relationship

```
Brand
  |
  +-- Product
        |
        +-- Product Variant
        |     |
        |     +-- Product Specifications
        |
        +-- Merchant Listing
```

```
                |
                +-- Price History
                +-- Inventory Snapshot
                +-- Store Availability
```

This design prevents duplicate product pages when multiple sellers list the same product using different names.

16 API Design

The backend should use versioned endpoints such as `/api/v1`.

Method	Endpoint	Purpose
GET	`/api/v1/products`	Search and filter canonical products.
GET	`/api/v1/products/{id}`	Retrieve product information and specifications.
GET	`/api/v1/products/{id}/listings`	Retrieve merchant prices and availability.
GET	`/api/v1/products/{id}/price-history`	Retrieve historical price data.
GET	`/api/v1/stores/nearby`	Find stores near a geographic coordinate.
GET	`/api/v1/deals`	Retrieve active and verified offers.
POST	`/api/v1/builds`	Create a PC build.
PUT	`/api/v1/builds/{id}`	Update a saved build.
POST	`/api/v1/builds/validate`	Validate selected component compatibility.
POST	`/api/v1/alerts`	Create a price or stock alert.
POST	`/api/v1/reviews`	Submit a product review.
POST	`/api/v1/merchant/listings`	Create a merchant listing.
POST	`/api/v1/merchant/imports`	Submit a merchant feed or file import.

Method	Endpoint	Purpose
POST	`/api/v1/outbound-clicks`	Record an external seller-link click.

17 Search and Product Matching

17.1 Search Requirements

The search system should understand variations such as:

- RTX5070
- RTX 5070
- ASUS 5070 Dual
- Product model numbers
- Common misspellings
- Abbreviations
- Specification-based queries

The first version may use PostgreSQL full-text search and trigram similarity. A larger platform may later introduce Typesense, Meilisearch, Elasticsearch, or OpenSearch.

17.2 Product Matching

Merchant listings should be matched to canonical products using:

- Brand
- Manufacturer model number
- Universal identifiers, where available
- Product category
- Key specifications
- Normalized title
- Fuzzy text similarity
- Administrative review

Automatic matching should produce a confidence score. Low-confidence matches should be reviewed by an administrator before publication.

18 Data Collection and Quality

18.1 Data Sources

Product information may be obtained through:

- Official manufacturer information
- Merchant APIs
- Merchant CSV or XML feeds
- Affiliate networks
- Public structured metadata
- Merchant dashboard submissions
- Permitted data collection
- Manual administrative entry

18.2 Quality Controls

Each listing should maintain:

- Source
- Last collected time
- Last successful verification
- Matching confidence
- Availability confidence
- Error state
- Merchant verification status

The system should automatically flag:

- Unusually large price changes
- Missing or invalid prices
- Duplicate listings

- Broken seller links
- Expired offers
- Conflicting specifications
- Listings not updated within an acceptable period

19 Merchant Portal

The merchant portal will allow approved sellers to manage their presence on the platform.

Merchant features should include:

- Business verification
- Store-location management
- Manual listing creation
- Bulk CSV import
- API or feed integration
- Price and stock updates
- Offer submission
- Referral-click analytics
- Listing-error reports
- Subscription and promotion management

Paid placements must be clearly marked as sponsored. Organic ranking should remain distinguishable from promotional ranking.

20 Security and Privacy

20.1 Security Measures

The platform should implement:

- Secure authentication through Supabase
- Role-based access control
- PostgreSQL Row-Level Security
- Server-side input validation

- API rate limiting
- Secure password and session handling
- CSRF protection where applicable
- Content sanitization
- Audit logging
- Merchant API-key protection
- Restricted administrative endpoints
- Dependency and vulnerability monitoring

20.2 Privacy Principles

The platform should:

- Collect only necessary personal information.
- Request permission before using precise location.
- Allow users to remove saved data and accounts.
- Avoid exposing private merchant or user information.
- Document analytics and tracking practices.
- Store sensitive secrets outside source code.

21 Review and Trust System

The review system should distinguish among:

- Customer reviews
- Verified-purchase reviews, where confirmation is possible
- Editorial reviews
- Expert reviews
- Imported external ratings

Reviews should support:

- Numerical ratings
- Written comments

- Advantages and disadvantages
- Helpful votes
- Abuse reporting
- Moderation status

AI-generated summaries may be used to summarize existing reviews, but they should be clearly labeled and should never be presented as genuine customer reviews.

22 Business Model

Potential revenue sources include:

- Affiliate commissions
- Cost-per-click merchant referrals
- Merchant subscriptions
- Sponsored listings
- Featured deals
- Merchant analytics
- Premium user features
- Clearly labeled advertising

Possible premium features may include:

- Advanced price alerts
- Longer price-history access
- Advanced build analysis
- Multiple saved comparison lists
- Detailed performance estimates
- Removal of general advertising

Commercial relationships must not secretly influence comparison results or recommendations.

23 Implementation Plan

23.1 Phase 1: Foundation and Catalog

- Establish the frontend, backend, and database structure.
- Implement authentication and roles.
- Develop the category and product catalog.
- Create structured product specifications.
- Implement merchants and merchant listings.
- Build the administrative product-matching interface.
- Implement product search and filtering.

23.2 Phase 2: Comparison and Price Tracking

- Add merchant price comparison.
- Add availability indicators.
- Record price history.
- Add wishlists.
- Add price and stock alerts.
- Implement outbound seller links.

23.3 Phase 3: PC Builder

- Implement saved PC builds.
- Add component-selection flows.
- Create compatibility rules.
- Add wattage estimation.
- Add warnings and recommendations.
- Create shareable build pages.

23.4 Phase 4: Locations and Deals

- Add store locations and maps.
- Add distance-based searching.

- Create the deals center.
- Add merchant offer submissions.
- Implement deal expiry and verification.

23.5 Phase 5: Visualization

- Create a two-dimensional build view.
- Add component-placement guidance.
- Add case-clearance visualization.
- Introduce selected 3D models.
- Add RGB and appearance customization.

23.6 Phase 6: Platform Expansion

- Add laptops and gadgets.
- Add gaming-product comparison.
- Add system-requirement matching.
- Add verified game-rental listings.
- Expand merchant and geographic coverage.

24 Minimum Viable Product

The recommended MVP should contain:

- PC component categories
- A limited number of trusted merchants
- Canonical product pages
- Structured specifications
- Search and filters
- Seller-price comparison
- Basic price history
- Wishlists
- Price alerts

- PC builder
- Core compatibility checks
- Saved and shareable builds
- Store locations
- Manually verified deals
- Administrative product-matching tools

The MVP should prioritize data accuracy and usability over the number of products.

25 Testing Strategy

25.1 Testing Types

The project should include:

- Unit testing
- API integration testing
- Database testing
- Compatibility-rule testing
- End-to-end testing
- Accessibility testing
- Performance testing
- Security testing
- Data-import testing
- Cross-browser and responsive testing

25.2 Critical Test Scenarios

Critical scenarios include:

- Matching differently named listings to the same product
- Rejecting incompatible CPU and motherboard combinations
- Detecting insufficient power-supply capacity
- Detecting GPU and case-clearance problems

- Expiring outdated deals
- Handling missing merchant prices
- Preventing unauthorized merchant modifications
- Preventing duplicate price-history entries
- Maintaining correct totals in a PC build

26 Risks and Mitigation

Risk	Impact	Mitigation
Outdated prices	Loss of customer trust	Display timestamps, refresh frequently, and flag stale listings.
Incorrect stock status	Customers may visit stores unnecessarily	Use availability confidence and advise users to contact the store.
Duplicate products	Confusing comparison results	Use canonical products, model numbers, matching scores, and review tools.
Incorrect specifications	Invalid comparisons and compatibility results	Use authoritative sources and administrative verification.
Unauthorized data collection	Legal or technical problems	Prefer APIs, feeds, permissions, and compliant collection policies.
Fake reviews	Reduced trust	Use moderation, abuse detection, and verified-review indicators.
Incorrect compatibility advice	Customers may purchase unsuitable components	Test rules, cite assumptions, and use warnings for uncertain cases.
Merchant bias	Unfair recommendations	Separate sponsored content from organic results.

Risk	Impact	Mitigation
Excessive 3D-development effort	Delayed core platform	Launch 2D visualization first and limit initial 3D coverage.
Rapid external website changes	Broken imports	Use modular connectors, monitoring, error reporting, and merchant feeds.

27 Success Metrics

The platform may track the following indicators:

- Number of canonical products
- Number of active merchant listings
- Percentage of recently verified prices
- Product-matching accuracy
- Search success rate
- Number of saved PC builds
- Compatibility-error detection rate
- Price-alert engagement
- Outbound merchant clicks
- Returning-user percentage
- Merchant retention
- Reported listing-error rate

Data quality should be treated as a primary success metric rather than measuring only traffic or catalog size.

28 Recommended Team Structure

A small initial team may include:

- Product owner or project manager
- Frontend developer

- Backend developer
- Database and data-engineering contributor
- UI/UX designer
- Product-data researcher
- Quality-assurance contributor

For an individual or student project, these responsibilities can be combined and implemented incrementally.

29 Final Evaluation

The proposed platform is technically achievable and commercially promising. Its value does not come merely from displaying products from different websites. Its real value comes from organizing unreliable and fragmented information into a trustworthy decision-support system.

The most important technical challenge is the creation of a clean, standardized, and continuously updated product database. Product normalization, seller-listing matching, price freshness, stock confidence, and compatibility accuracy are more important than advanced visual effects during the initial stages.

The PC simulator is a valuable differentiating feature, but it should be built on top of a reliable product catalog and compatibility engine. A two-dimensional visualization should be implemented first, followed by limited three-dimensional support for popular products.

The recommended launch strategy is to begin with PC components, trusted merchants, price comparison, compatibility checking, store discovery, and verified deals. Laptops, gadgets, games, rentals, and a complete 3D simulation can be introduced after the core platform has demonstrated accuracy and user value.

30 Conclusion

TechHub aims to become a central technology-shopping assistant that helps users understand products before purchasing them. It will combine product discovery, seller comparison, compatibility validation, local store search, promotional offers, and PC visualization within a single platform.

React or Next.js, FastAPI, and Supabase provide an appropriate technical foundation. The system should be developed through carefully controlled phases, beginning with a focused minimum viable product.

If the platform maintains accurate data, transparent recommendations, and clear seller information, it can provide meaningful value to customers while generating qualified traffic and analytics for merchants.

A Suggested Technology Stack

Layer	Technology	Purpose
Frontend	Next.js, React, TypeScript	User interface, server rendering, routing, and SEO.
Styling	Tailwind CSS	Responsive design and reusable styling.
Server state	TanStack Query	Fetching, caching, and synchronizing API data.
Client state	Zustand	Managing PC-builder and interface state.
Backend	FastAPI	APIs, business logic, validation, and integrations.
Database	Supabase PostgreSQL	Relational product, merchant, listing, and build data.
Authentication	Supabase Auth	User registration, sessions, and identity management.
Storage	Supabase Storage	Product, review, merchant, and editorial media.
Background jobs	Celery, Dramatiq, or ARQ	Imports, updates, matching, and notifications.
Queue/cache	Redis	Task queuing, caching, and rate limiting.
Search	PostgreSQL initially	Full-text, trigram, and filtered product searching.
Advanced search	Typesense or Meilisearch	Optional large-scale typo-tolerant searching.
Maps	Mapbox or Google Maps	Store locations, distance, and directions.

Layer	Technology	Purpose
Charts	Recharts or Chart.js	Price-history and analytics visualization.
3D visualization	React Three Fiber	Future interactive PC visualization.
Testing	Pytest, Vitest, Playwright	Backend, frontend, integration, and end-to-end tests.
Deployment	Docker and managed hosting	Consistent deployment and service isolation.

B Suggested Repository Structure

Listing 3: Possible project repository structure

```
techhub/
|
+-- frontend/
|   +-- src/
|   |   +-- app/
|   |   +-- components/
|   |   +-- features/
|   |   |   +-- products/
|   |   |   +-- comparison/
|   |   |   +-- pc-builder/
|   |   |   +-- deals/
|   |   |   +-- stores/
|   |   |   +-- authentication/
|   |   +-- services/
|   |   +-- stores/
|   |   +-- types/
|   |   +-- utils/
|   +-- tests/
|
+-- backend/
|   +-- app/
|   |   +-- api/
|   |   +-- models/
|   |   +-- schemas/
|   |   +-- services/
```

```
|   |   +-- repositories/
|   |   +-- compatibility/
|   |   +-- ingestion/
|   |   +-- workers/
|   |   +-- security/
|   |   +-- core/
|   +-- tests/
|
+-- database/
|   +-- migrations/
|   +-- seeds/
|   +-- policies/
|
+-- docs/
|   +-- api/
|   +-- architecture/
|   +-- product-rules/
|
+-- docker/
+-- README.md
```

C Example User Journey

1. The user searches for a gaming processor.
2. The platform displays matching canonical products.
3. The user opens a product and reviews its specifications.
4. The user compares prices from verified merchants.
5. The user adds the processor to a new PC build.
6. The system displays compatible motherboards.
7. The user selects the remaining components.
8. The system checks sockets, dimensions, connectors, and wattage.
9. The user views a visual representation of the build.
10. The platform shows the total price across different merchants.
11. The user saves and shares the build.
12. The user selects a seller or nearby store.

13. The platform records the referral and redirects the user.